DevSync — API Documentation

Secure AI Project Collaborator · Android app · Flutter + Firebase + Cloud Functions + LLM API

Document version	0.1 (draft)
Date	27 October 2026
Scope	All 13 core features (64 screens, 147 operations)
Source documents	Team Plan, Comprehensive Project Analysis (20 Oct 2026), Core Features (20 Oct 2026), Stitch Prompts v2, Literature Review v1.1
Planned owners (per team plan)	Rana (API documentation, task T076), Mohraeel (Cloud Functions and schema)
Team plan target	6 November 2026

How to read this document

Every contract carries a status tag:

- **[Defined]** is stated in the project documents.
- **[Proposed]** is a contract drafted here from the project documents. It must be confirmed with Mohraeel before it is frozen.
- **[Unapproved]** is a Should/Could item, or something outside the approved plan. It is included for completeness only.

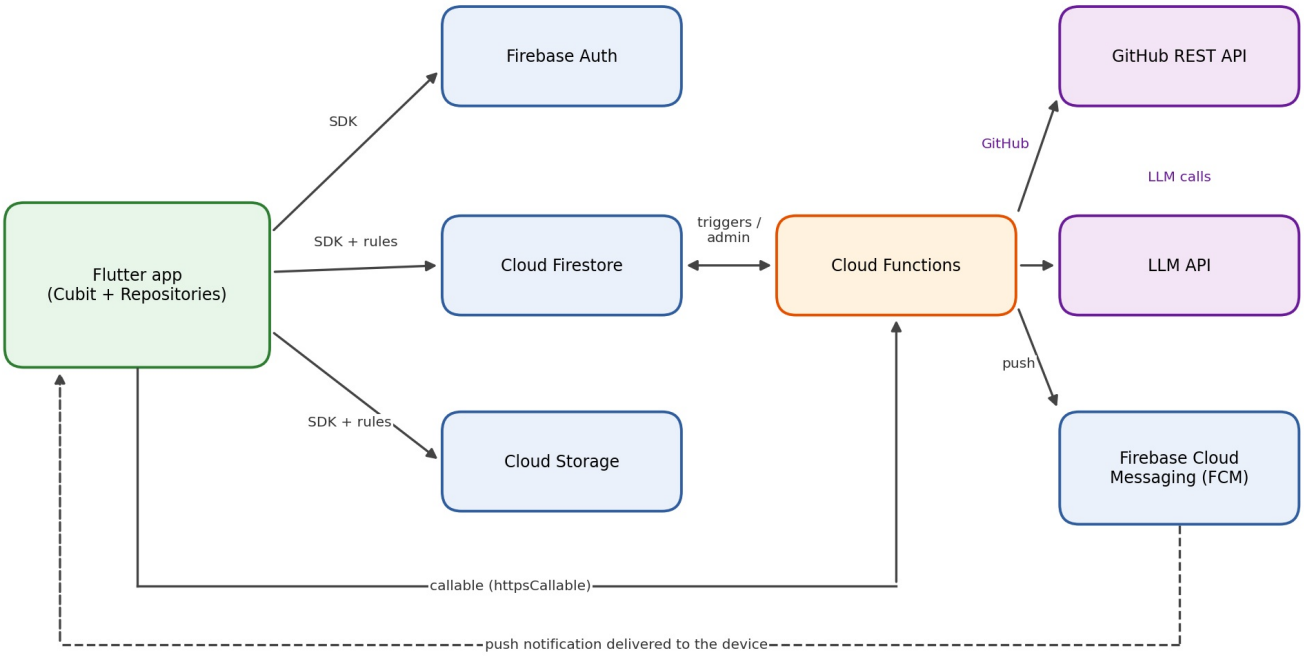
> If `docs/schema.md` and Section 5 disagree, `docs/schema.md` is the source of truth. The Firestore schema freeze date in the team plan is 24 October 2026.

Contents

1. Architecture overview
2. Feature to API matrix
3. Conventions
4. Cloud Functions reference
5. Firestore data model
6. Cloud Storage
7. Repository layer (Dart)
8. Feature reference (13 features)
9. Cross-cutting services
10. Security rules and privacy
11. Error handling matrix
12. Performance targets
13. Open decisions
14. Appendices

1. Architecture overview

DevSync has **no custom REST server**. The backend is built from Firebase services. The Flutter app talks to them through the SDK, behind a Repository layer. The only code that talks to third-party APIs is Cloud Functions.



Surface	Used for	Called by
Firebase Auth	Email and Google sign-in, sessions, GitHub OAuth link	App SDK
Cloud Firestore	All application data, guarded by security rules	App (via repositories) and Functions (admin)
Cloud Storage	CV and profile photo	App SDK
Callable Cloud Functions	GitHub import, AI explanation, pitch, project setup, roles, chatbot	App
Firestore-triggered Functions	Notifications (FCM + in-app list)	Firestore events
Firebase Cloud Messaging	Push notifications	Functions, to the device
Crashlytics	Crash-free session monitoring	App

Rules that shape every API (NFR-04): API keys and secrets (GitHub, LLM) live in Cloud Functions secrets. They are never in the app code or the repository. The app never calls GitHub or the LLM directly.

2. Feature to API matrix

#	Feature	Priority	Auth	Firestore	Storage	Callable Functions	FCM
1	Splash and Onboarding	Must	session check	none	none	none	none
2	Authentication (with Guest mode)	Must	yes	`users`	none	none	none
3	Profile (with Profile Completion)	Must	yes	`users`, `publicIds`	CV, photo	none	none
4	GitHub Integration	Must	GitHub provider	`users/*/githubData`, `detectedSkills`, `private`	none	`githubProxy`	none
5	AI Match	Must	yes	`matches`, `feedback`	none	`llmExplain`	none
6	Project Discovery	Must	optional (Guest read)	`projects`, `users`, `saved`	none	none	none
7	Community	Should / Could	yes (write)	`posts` (proposed)	none	none	none
8	Project Details	Must	yes (actions)	`projects`, `requests`, `saved`	none	`llmPitch`	via trigger
9	Create Project	Must (AI assist: Should)	yes	`projects`	none	`llmProjectSetup`, `llmRoles`	none
10	My Teams	Must (Tasks), Should (Chat)	yes	`teams`, `tasks`	none	none	via trigger
11	My Projects	Must	yes	`projects`, `requests`, `teams`	CV read	none	via trigger
12	Prompt Evaluation	Must	yes	`feedback`, `prompts`, `config`	none	none	none
13	Chatbot	Could (Pro-locked)	yes	none	none	`chatbotAsk` (unapproved)	none

3. Conventions

3.1 Authentication and Guest mode

- Every Cloud Function requires a signed-in Firebase user. A **Guest** has no Firebase user, so Guests cannot call functions.
- The Firebase SDK attaches the ID token to Firestore, Storage and callable requests automatically.
- Guests are limited to reading public data. The permission table is in Feature 2.
- Any action that needs an account goes through one function, `requireAuth(context, action)`. It opens the *Login Required* bottom sheet, saves `pendingRoute`, and returns the user to the same screen after login.

3.2 Calling a callable function (Dart)

```dart
final callable = FirebaseFunctions.instance.httpsCallable('llmExplain');
try {
  final res = await callable.call({
    'projectId': 'p_001',
    'score': 82,
    'matchedSkills': ['Flutter', 'Dart'],
    'missingSkills': ['REST API'],
  });
  final data = Map<String, dynamic>.from(res.data);
} on FirebaseFunctionsException catch (e) {
  // e.code, e.message, e.details  (see 3.3)
}
```

The function region is not defined in the plan. Set it once in the DI container (`FirebaseFunctions.instanceFor(region: ...)`).

3.3 Error model

Functions throw standard Firebase `HttpsError` codes. The app maps each code to a user-facing message. The app never shows raw errors.

Code	Meaning in DevSync	App behavior
`unauthenticated`	No valid session (Guest or expired)	Login Required sheet or re-login
`permission-denied`	Caller is not allowed (for example, not the project owner)	Clear message; no retry
`invalid-argument`	Missing or malformed input	Validation message
`not-found`	Project, profile or role does not exist	"No longer available" message
`already-exists`	Duplicate resource	"Already done" message
`failed-precondition`	Required state is missing (GitHub not connected, project not open)	Guide the user to fix it
`resource-exhausted`	GitHub rate limit reached, or per-user daily AI limit reached	Message and fallback; no retry loop
`deadline-exceeded`	LLM timeout	Fallback (see 3.4)
`unavailable`	GitHub or LLM is down, or the AI output was invalid	Fallback (see 3.4)
`internal`	Unexpected error	Generic message; logged to Crashlytics

`details` may carry `{ reason: string, retryAfterSeconds?: number }` **[Proposed]**.

3.4 AI conventions

- **Graceful degradation (NFR-11).** If the LLM is slow or down, the user never sees an empty screen. `llmExplain` returns rule-based or cached text with `source: "fallback"`. Where there is no fallback text (pitch, project setup, roles), the function throws and the app moves to manual input with a clear message (UC-04).
- **Editable output.** All generated content (project setup, roles, pitch) is editable before it is published or sent. Nothing is published automatically without the owner's approval.
- **Explanation first.** A match score never appears alone. It always comes with matched skills, missing skills and an explanation.
- **Prompt versioning.** Every AI function reads the active version from `config/prompts.activeVersion`. It loads the template from `prompts/{version}` and returns `promptVersion` in its response. Feedback is stored with that version (see Feature 12).
- **Cost control.** Each user has a daily limit on AI requests, tracked by a counter in Firestore. The number is to be set by the team. A short timeout applies to every LLM call.
- **Validation.** Functions that return structured JSON (`llmProjectSetup`, `llmRoles`) validate the shape before returning it.

3.5 Common types

Type	Format
`uid`	Firebase Auth user ID
`publicId`	Short unique code such as `DS-8K2Q`, generated once per user
`projectId`	Firestore auto-ID
`requestId`	`{projectId}_{applicantUid}` (composite ID prevents duplicate requests)
Skill name	Normalized name from `skills_catalog.dart` (for example `Flutter`, `Dart`, `Firebase`, `REST API`), with aliases
Timestamp	Firestore `Timestamp` in documents; ISO-8601 string in function responses **[Proposed]**

3.6 Pagination

`getProjects` uses `limit(10)` and `startAfter(lastDocument)`. Composite filters need composite indexes declared in `firestore.indexes.json`. If an index is missing, the Firestore console shows an error with a link to create it.

---

4. Cloud Functions reference

Cloud Functions run on Node.js / TypeScript. All callable functions require authentication.

Function	Type	Priority	Purpose	Status
`githubProxy`	Callable	Must	Store the GitHub token, sync repos/languages/activity, disconnect	Defined (behavior); input/output Proposed
`llmExplain`	Callable	Must	Explanation and learning tip per missing skill	Defined (behavior); input/output Proposed
`llmPitch`	Callable	Must	Draft the collaboration request message	Defined (name); input/output Proposed
`llmProjectSetup`	Callable	Must in plan (T068), Should in Core Features	Turn a two-sentence idea into a project setup	Defined (name); input/output Proposed
`llmRoles`	Callable	Must in plan (T012), Should for full role generator	Generate required roles	Defined (name); input/output Proposed
`chatbotAsk`	Callable	Could	In-app assistant	Unapproved
Notification triggers	Firestore triggers	Must	FCM push and in-app notification record	Defined (behavior); names Proposed

4.1 `githubProxy`

Imports the developer's GitHub data. The GitHub OAuth token is stored **server-side only**, in `private/{uid}`. Client rules deny all reads and writes on `private`.

Request (one function, three actions) **[Proposed]**

Field	Type	Required	Description
`action`	string	yes	`storeToken`, `sync` or `disconnect`
`accessToken`	string	only for `storeToken`	GitHub provider access token from Firebase Auth (scope `read:user`). Sent once; never persisted in the app (NFR-07)
`force`	boolean	no (`sync`)	Skip the cache. The app throttles how often it is allowed

Behavior

Action	What the function does
`storeToken`	Validates the token with GitHub, saves it to `private/{uid}`, sets `users/{uid}.githubConnected = true`
`sync`	Using authenticated requests (higher rate limit), fetches repos, languages and activity. Writes `users/{uid}/githubData` (`reposSummary`, `languages`, `activity`, `lastSync`). Returns the cached copy if `lastSync` is less than about one hour old and `force` is false
`disconnect`	Deletes `githubData`, `detectedSkills` and the stored token, and sets `githubConnected = false`

Response (`sync`) **[Proposed]**

```
{
  "fromCache": false,
  "lastSync": "2026-11-08T10:15:00Z",
  "reposCount": 14,
  "topLanguages": [{ "name": "Dart", "share": 0.52 }, { "name": "TypeScript", "share": 0.21 }],
  "activity": { "lastActive": "2026-11-06" }
}
```

The shape of `activity` is still to be fixed. It needs at least the last-active date for Insights and for skill recency.

Skill inference (rule-based, no LLM). Languages become skills. Repo topics and names map to skills through a lookup table (a repo with the topic `flutter` implies `Flutter`). Each inferred skill is written to `users/{uid}/detectedSkills/{id}` with `skill`, `confidence` (based on the number of repos and recency of activity), `evidence` (repo names), `lastActive` and `status: "pending"`. Whether this step runs in the app or in the function is still open (see Section 13).

Errors

Code	When	App behavior
`failed-precondition`	GitHub not connected or token revoked	Offer to reconnect
`resource-exhausted`	GitHub rate limit reached	Clear message; the user continues with manual data (UC-01 alternate path)
`unavailable`	GitHub is down	Same as above

4.2 `llmExplain`

Explains a match in natural language and gives a short learning tip for every missing skill. The score and skill lists come from the client-side `MatchingService`.

Request [Proposed]

Field	Type	Required	Description
`projectId`	string	yes	Project being explained
`score`	number (0–100)	yes	Match score from `MatchingService`
`matchedSkills`	string[]	yes	Skills the user and project share
`missingSkills`	string[]	yes	Skills the project needs that the user lacks

The function loads the user's profile and the project from Firestore itself. It does not trust profile data sent by the client.

Response [Proposed]

```
{
  "explanation": "Your Flutter and Dart skills match the core stack, and your recent GitHub activity includes Flutter work.",
  "complementarySkills": ["Firebase"],
  "learningTips": [
    { "skill": "REST API", "tip": "Build a small Flutter screen that reads from a public REST API." }
  ],
  "promptVersion": "v2",
  "source": "llm",
  "fallbackReason": null
}
```

Field	Notes
`source`	`llm`, `cached` or `fallback`
`fallbackReason`	`timeout`, `unavailable` or `daily_limit` when `source` is `fallback`; otherwise `null`. The app uses it to show a short notice

The explanation and `promptVersion` are cached in `matches/{uid}/items/{projectId}`. On LLM failure or timeout the function returns ready-made rule-based text. It does not throw.

4.3 `llmPitch`

Drafts the collaboration message for a Request to Join. The user always reviews and edits it before sending.

Request [Proposed]: `{ "projectId": string, "role": string }`. `role` is one of `projects/{projectId}.roles[].name`.

Response [Proposed]: `{ "pitch": string, "promptVersion": string }`

Errors: `not-found` (project or role), `failed-precondition` (project is not open), `resource-exhausted` (daily AI limit), `deadline-exceeded` or `unavailable` (the app lets the user write the message manually).

4.4 `llmProjectSetup`

Turns a two-sentence idea into an editable project setup (UC-04, Create Project).

Request [Proposed]: `{ "idea": string }`. The maximum length is to be set by the team.

Response [Proposed]

```
{
  "title": "Study Group Finder",
  "description": "A mobile app that helps students find study partners by subject and level.",
  "techStack": ["Flutter", "Firebase"],
  "difficulty": "beginner",
  "roles": [
    { "name": "Mobile Developer", "requiredSkills": ["Flutter", "Dart"], "count": 2 },
    { "name": "Backend Developer", "requiredSkills": ["Firebase"], "count": 1 }
  ],
  "suggestedTasks": ["Set up the Firebase project", "Design the profile screen"],
  "promptVersion": "v1"
}
```

The function validates the JSON shape and should normalize skill names to the skills catalog before returning. If the model output is invalid, the function throws `unavailable` with `details.reason = "invalid_ai_output"`, and the app falls back to manual input.

4.5 `llmRoles`

Generates the required team roles for a project. Used by the Role Generator screen and by Missing Role Alert.

Request [Proposed]: `{ "title": string, "description": string, "techStack": string[], "teamSize": number }` (`teamSize` is optional).

Response [Proposed]: `{ "roles": [{ "name": string, "requiredSkills": string[], "count": number }], "promptVersion": string }`

The owner can edit, add or delete roles after generation. Errors are the same as `llmProjectSetup`.

4.6 `chatbotAsk` — [Unapproved]

The chatbot is Could priority, outside the approved plan, and locked in the Pro flavor. This contract is a placeholder.

Request: `{ "message": string, "history": [{ "role": "user" or "assistant", "text": string }] }` (`history` is optional and capped)

Response: `{ "reply": string, "suggestions": string[], "deepLink": string, "promptVersion": string, "source": "llm" or "fallback" }`

Replies use the user's profile and skills as context. The function enforces a per-user request cap. If the LLM is down it returns a clear fallback message, never an empty screen. Chat history lives in the app by default, so "clear history" is local **[Proposed]**.

4.7 Notification triggers

These are Firestore-triggered functions, not callable. Each one sends an FCM push **and** writes a record to `notifications/{uid}/items/{id}`.

Trigger [names Proposed]	Fires when	Recipient	type
`onRequestCreated`	`requests/{requestId}` is created	Project owner	`request_received`
`onRequestStatusChanged`	`requests/{requestId}.status` moves from `pending` to `accepted` or `rejected`	Applicant	`request_accepted` or `request_rejected`
`onTaskWritten`	A task is created, or its assignee or status changes	Assignee	`task_assigned` or `task_updated`

Invitation and message notifications are Should and [Unapproved].

FCM data payload [Proposed]. All values are strings.

```
{ "type": "request_accepted", "refId": "p_001_u_abc", "projectId": "p_001" }
```

- The app router reads `type` and `refId` and opens the right screen (request, acceptance, task).
- The app handles three states: foreground, background, and terminated (`getInitialMessage`).
- The target delivery time is **10 seconds or less** on real devices (NFR-02).
- On Android 13 and later, the notification permission is requested when it is first needed, not at app launch.
- Device tokens are stored in `users/{uid}.fcmTokens[]` and refreshed when they change.

Notification record `notifications/{uid}/items/{id}` **[Defined]:** `type`, `refId`, `title`, `body`, `read`, `createdAt`.

---

5. Firestore data model

Source: Team Plan (schema freeze target 24 October 2026) and Core Features. Fields marked *(proposed)* are needed by a feature but are not in the frozen schema. Each collection has one Dart model in `lib/data/models` with `fromMap`, `toMap` and `copyWith`.

Path	Fields	Written by	Notes
`users/{uid}`	`name`, `bio`, `track`, `level`, `languages[]`, `skills[]`, `softSkills[]`, `interests[]`, `background`, `cvUrl`, `portfolioUrl`, `linkedinUrl`, `behanceUrl`, `githubConnected`, `publicId`, `fcmTokens[]`, `createdAt`, `photoUrl` *(proposed)*	Owner	Base document is created at sign-up
`users/{uid}/githubData`	`reposSummary`, `languages`, `activity`, `lastSync`	`githubProxy`	Client reads only
`users/{uid}/detectedSkills/{id}`	`skill`, `confidence`, `evidence`, `lastActive`, `status` (`pending`, `accepted`, `rejected`)	Inference step; the user updates `status`	AI-inferred skills are never shown as verified facts
`users/{uid}/saved/{projectId}`	saved project reference	Owner	Save and Favorites are merged into one feature, "Saved"
`private/{uid}`	GitHub token	`githubProxy` only	Client has no read or write access
`publicIds/{id}`	reserved user ID	Profile creation (transaction)	Guarantees a unique `publicId`
`projects/{projectId}`	`title`, `description`, `purpose` (`learning`, `real`), `techStack[]`, `difficulty`, `teamSize`, `status` (`draft`, `open`, `full`, `completed`), `ownerId`, `roles[{name, requiredSkills[], count, filled}]`, `expectedCommitment`, `mentoring`,	Owner	Readable by everyone,

	`createdAt`, plus `minLevel` and `expectedEndDate` *(proposed)*		including Guests
`requests/{projectId_applicantId}`	`projectId`, `applicantId`, `ownerId`, `role`, `pitch`, `status` (`pending`, `accepted`, `rejected`), `createdAt`	Applicant creates; owner changes `status`	Composite ID blocks duplicates
`teams/{projectId}`	`memberIds[]`	Accept transaction	Created on the first accepted request
`teams/{projectId}/members/{uid}`	member role	Accept transaction	Shown in Team Members
`teams/{projectId}/tasks/{taskId}`	`title`, `description`, `assigneeId`, `status` (`todo`, `inProgress`, `inReview`, `done`), `priority`, `deadline`, `createdBy`	Team members	Members only
`matches/{uid}/items/{projectId}`	`score`, `matchedSkills[]`, `missingSkills[]`, `explanation`, `promptVersion`	Client (scores), `llmExplain` (explanation)	Cache and Match History
`feedback/{id}`	`uid`, `projectId`, `helpful`, `clarity` (1–5), `reason`, `promptVersion`	Author	Input to Prompt Evaluation
`prompts/{version}`	`name`, `template`, `createdAt`	Admin	Versions `v1`, `v2`, ...
`config/prompts`	`activeVersion`	Admin	Read by the AI functions
`notifications/{uid}/items/{id}`	`type`, `refId`, `title`, `body`, `read`, `createdAt`	Functions	The user can update `read`

Example: project document

```
{
  "title": "Study Group Finder",
  "description": "Find study partners by subject and level.",
  "purpose": "learning",
  "techStack": ["Flutter", "Firebase"],
  "difficulty": "beginner",
  "teamSize": 4,
  "status": "open",
  "ownerId": "u_owner",
  "roles": [
    { "name": "Mobile Developer", "requiredSkills": ["Flutter", "Dart"], "count": 2, "filled": 1 },
    { "name": "QA Engineer", "requiredSkills": ["Testing"], "count": 1, "filled": 0 }
  ],
  "expectedCommitment": "6 hours/week",
  "mentoring": true
}
```

A role where `filled < count` triggers the **Missing Role Alert**.

Example: request document (`requests/p_001_u_abc`)

```
{
  "projectId": "p_001",
  "applicantId": "u_abc",
  "ownerId": "u_owner",
  "role": "Mobile Developer",
  "pitch": "I have built two Flutter apps and want to learn Firebase on a real team.",
  "status": "pending"
}
```

6. Cloud Storage

Path	Content	Rules
`cvs/{uid}/cv.pdf`	User CV	PDF only, 5 MB maximum, validated by type. The owner uploads and reads. The owner of a project the user applied to can read it (mechanism to be decided with Mohraeel)
Profile photo	Avatar image	Path not defined in the plan. Proposed: `avatars/{uid}/avatar.jpg` *(proposed)*

CV upload and display are in scope. AI analysis of the CV is **not** in the plan (future work).

7. Repository layer (Dart)

Each module has an abstract repository, a **Mock** implementation (used from day one) and a **Firebase** implementation. `get_it` switches between them in one place (`useMock ? Mock… : Firebase…`).

```
abstract class AuthRepository {
  Future<void> signUp(String name, String email, String password);
  Future<void> sendVerification();
  Future<void> login(String email, String password);
  Future<void> googleSignIn();
  Future<void> sendReset(String email);
  Future<void> logout();
  Future<void> deleteAccount();
  Stream<AuthUser?> authState();
}

abstract class ProfileRepository {
  Future<UserProfile> get(String uidOrPublicId);
  Future<void> createOrUpdate(UserProfile profile);
  Future<String> uploadCv(File pdf);
}

abstract class ProjectRepository {
  Future<List<Project>> getProjects(
      ProjectFilters filters, ProjectSort sort, int limit, DocumentSnapshot? startAfter);
}
```

The signatures above are **[Defined]** by the team plan (parameter types are illustrative). The other repositories are **[Proposed]**:

Repository	Feature	Main methods
`GithubRepository`	4	`connect`, `sync`, `disconnect`, `watchInsights`, `watchDetectedSkills`, `reviewSkill`
`MatchRepository`	5	`getMatch`, `watchHistory`, `explain` (calls `llmExplain`)
`RequestRepository`	8, 11	`send`, `watchStatus`, `watchIncoming`, `accept`, `reject`
`TeamRepository`	10	`watchMyTeams`, `getMembers`, `leave`
`TaskRepository`	10	`watchTasks`, `create`, `update`, `delete`
`FeedbackRepository`	5, 12	`submit`, `getPromptReport`
`NotificationRepository`	9.1	`watch`, `markRead`, `saveToken`

---

8. Feature reference

Every feature lists its screens, its operations (numbered as in the Core Features document), and the API each operation uses. The operation counts add up to **147**.

#	Feature	Screens	Operations	Priority
1	Splash and Onboarding	4	6	Must
2	Authentication (with Guest mode)	5	10	Must
3	Profile (with Profile Completion)	10	21	Must
4	GitHub Integration	3	8	Must
5	AI Match	3	12	Must
6	Project Discovery	7	13	Must
7	Community	3	10	Should / Could
8	Project Details	3	12	Must
9	Create Project	8	14	Must (AI assist: Should)
10	My Teams	7	12	Must (Tasks), Should (Chat)
11	My Projects	8	13	Must
12	Prompt Evaluation	2	7	Must
13	Chatbot	1	9	Could
	Total	**64**	**147**	

Success states, empty states and bottom sheets are states of the same Flutter page, so the real page count is below 64.

Feature 1 — Splash and Onboarding

Goal: a first impression that explains the idea in three slides, then sends the user to Auth or to Discover as a Guest.

Screens (4): Splash (logo, tagline "Connect. Collaborate. Build."), Onboarding 1 "Discover your fit", Onboarding 2 "Understand your match", Onboarding 3 "Build together" (Get Started and Continue as Guest).

Backend: none. This feature uses only the Auth session and local storage.

Routing at launch: session found → Discover. No session and first launch → Onboarding. Otherwise → Auth.

#	Operation	API / implementation	Notes

1	Check for an existing session, go straight to Discover	`FirebaseAuth` current user / `authStateChanges()`	`AuthGuard` (Malak)
2	Check the first-launch flag (Onboarding shows once)	`SharedPreferences` key `onboarding_seen`	Local
3	Next / swipe between slides	Local (`PageView`)	Three slides with dots
4	Skip	Local	Sets the flag
5	Get Started → Auth screen	Local navigation	
6	Continue as Guest → Discover (read-only)	`AuthCubit` state `Guest`	No Firebase user is created

Feature 2 — Authentication (with Guest mode)

Goal: secure sign-in, and the ability for a visitor to browse before signing up.

Screens (5, plus the Login Required bottom sheet as a state): Auth Entry, Create Account, Log In, Forgot Password, Reset Confirmation ("Check your inbox").

Sign-in methods: Email and Google (FR-01). GitHub is connected later from the profile (Feature 4). LinkedIn is only a profile link.

`AuthCubit` states: `Loading`, `Authenticated`, `Guest`, `Unauthenticated`, `Error`.

#	Operation	API / implementation	Notes
1	Sign up with email	`AuthRepository.signUp` → `createUserWithEmailAndPassword`, then create base doc `users/{uid}`	Fields: full name, email, password, confirm password, terms checkbox
2	Send email verification	`sendVerification` → `sendEmailVerification()`	A Resend button handles the "email not verified" state
3	Log in with email	`login` → `signInWithEmailAndPassword`	Handle wrong password, unverified email, no network
4	Sign in with Google	`googleSignIn` → `google_sign_in` + `signInWithCredential`	The SHA-1 and SHA-256 fingerprints of **each flavor** must be registered in Firebase; test on a real device
5	Forgot password (send reset link)	`sendReset` → `sendPasswordResetEmail`	
6	Resend reset link	`sendReset`	Resend Email button
7	Guest browsing (read-only)	`AuthCubit` = `Guest`; no API call	Reads are allowed by security rules on public collections
8	Login gate	`requireAuth(context, action)` → `LoginRequiredBottomSheet`	Saves `pendingRoute`, returns to the same screen after login
9	Log out	`logout` → `signOut()`	From Settings
10	Delete account	`deleteAccount` → `User.delete()` plus deletion of the user's data	From Settings. Firebase may require a recent login before deletion. The data-deletion mechanism is open (Section 13)

Guest permissions

Allowed (read only)	Requires login
Discover and browse projects	Join / Request to Join
Project Details (read)	Create Project
Community Feed (read)	Save / Express Interest
Public profiles	Notifications and Messages
Filters and search	Personal AI Match, Chatbot, Profile

Feature 3 — Profile (with Profile Completion)

Goal: a full picture of the developer. It is the input to AI Matching.

Screens (10): six setup steps (Basic info; Technical profile; Soft skills and interests; Documents and links; Connect accounts; Developer Snapshot), then Profile Completion, My Profile, Edit Profile, Connected Accounts. A **Profile Completion Banner** component appears on Discover Home and My Profile when the profile is incomplete.

Required fields (US-02): name, main track, level, and at least one language. Validation lives in the Cubit, not in the widget.

Data: photo, name, bio, unique user ID, main track (Mobile, Web, Backend, Data, AI, UI/UX, QA, DevOps), level (Beginner, Junior, Intermediate, Advanced), programming languages, technical skills, soft skills, interests, other technical background, CV (PDF), portfolio link, GitHub or Behance, LinkedIn link, and an optional free-text description of what the user wants to work on.

#	Operation	API / implementation	Notes
1	Edit basic info	`ProfileRepository.createOrUpdate` → `users/{uid}`	
2	Upload profile photo	Storage upload, then `users/{uid}.photoUrl`	Path and field *(proposed)*
3	Write or edit Bio	`createOrUpdate(bio)`	
4	Choose Main Track	`createOrUpdate(track)`	

5	Choose Level	`createOrUpdate(level)`	
6	Add or remove programming languages	`createOrUpdate(languages[])`	Element shape (name and level) to be frozen
7	Add or remove Skills with a level	`createOrUpdate(skills[])`	Chosen from `skills_catalog.dart` so matching works
8	Choose soft skills and interests	`createOrUpdate(softSkills[], interests[])`	
9	Write other technical background	`createOrUpdate(background)`	Courses, certificates, experience
10	Upload CV	`ProfileRepository.uploadCv` → Storage `cvs/{uid}/cv.pdf`, then `cvUrl`	PDF only, 5 MB maximum, type validation
11	Add Portfolio link	`createOrUpdate(portfolioUrl)`	
12	Add LinkedIn link	`createOrUpdate(linkedinUrl)`	
13	Connect GitHub / Behance	GitHub: see Feature 4. Behance: `behanceUrl`	Behance is link-only (Should)
14	Review the Snapshot and confirm	Read `users/{uid}` and `detectedSkills`	Shows the final summary with the AI-Detected Skills section
15	Edit the profile later	`createOrUpdate`	
16	View the Public Profile	`ProfileRepository.get` (public fields)	What other users see
17	Profile Completion indicator	Computed in the app, not stored	Target: 80% of testers complete their profile
18	Generate and show a unique user ID	Firestore transaction on `publicIds/{id}`, saved as `publicId`	Format `DS-8K2Q`, generated once
19	Show the Completion Banner if the profile is incomplete	App logic; a local flag controls when it returns after dismissal	
20	Open the Profile Completion screen	App navigation	Progress ring and checklist
21	Tap Add on a missing item, go to the right field, return, update the checklist	App navigation plus `createOrUpdate`	

Completion checklist (weights to be set by the team): name, track, level, at least one language, three or more skills, bio, CV, GitHub connected.

A connected-account status (GitHub, Behance) is shown only when the account is actually connected. If the user skips GitHub, they continue with manual data (UC-01).

Feature 4 — GitHub Integration

Goal: skills built on evidence from real work, not only on what the user typed.

Screens (3): Connect GitHub (what data is read, least privilege, Connect and Skip), Analysis in progress (animated progress), Review Detected Skills ("Add to my skills? Yes / No", with evidence). **GitHub Insights** is a section inside My Profile (top languages, activity, repo count), plus a Disconnect confirmation dialog.

#	Operation	API / implementation	Notes
1	Link the account	Firebase Auth GitHub provider (OAuth, scope `read:user`), then `githubProxy {action: "storeToken"}`	The token goes to the server only
2	Fetch repos, languages, commits and PRs	`githubProxy {action: "sync"}`	Authenticated requests with caching; progress shown in the app
3	Infer skills with evidence and confidence	Rule-based inference → `users/{uid}/detectedSkills` (`status: pending`)	See 4.1
4	Review detected skills (accept or reject)	Update `detectedSkills/{id}.status`	Accepted skills join the profile skills, tagged "from GitHub"
5	Show GitHub Insights	Read `users/{uid}/githubData`	Top languages, activity
6	Re-sync	`githubProxy {action: "sync", force: true}`	Time-throttled
7	Disconnect	`githubProxy {action: "disconnect"}`	Confirmation dialog; deletes the analysis data and the token
8	Handle rate limit or API failure	Catch `resource-exhausted` / `unavailable`	Clear message; the user continues with manual data

Product rules

- An inferred skill is **not a verified fact**. It is shown in a separate section and only the user can approve it.
- A GitHub activity score, if added later, measures verifiable activity only. It never labels a person as "trusted" or "not trusted".
- Only the data needed for matching is used. The token is never in the app code (NFR-04, NFR-07).

Feature 5 — AI Match

Goal: recommend projects **with an explanation**, not only a percentage, and tell the user what they are missing.

Screens (3): Why this project fits you, Skill Gap Analysis, Match History. Also a **Recommended for you** section on Discover Home and a **Feedback bottom sheet** (helpful / not helpful, plus a 1–5 clarity rating).

How matching works

1. A pure Dart class, `MatchingService`, has no Firebase dependency, so it is unit-testable and keeps working when the LLM is down.
2. **Input:** the user's skills, level and interests, and the project's requirements. **Output:** `score` (0–100), `matchedSkills[]`, `missingSkills[]` **[Proposed signature: `MatchResult match(UserProfile u, Project p)`]**.
3. Scoring ideas, in priority order: skill overlap (heaviest), level versus difficulty, interests, project purpose (a beginner prefers Learning). The weights are examples and must be tuned on test profiles.
4. `llmExplain` turns the result into natural language and adds a learning tip for each missing skill.

> Core Features describes recommendation generation as "Cloud Function + LLM". The Team Plan scores in the client-side `MatchingService` and uses the LLM only for the explanation. This document follows the Team Plan.

#	Operation	API / implementation	Notes
1	Generate recommendations from the skill profile	`MatchingService` over open projects → `matches/{uid}/items/{projectId}`	If the profile is incomplete, show "Complete your profile first" with a button to Profile Completion (UC-02 alternate path)
2	Show a list sorted by match score	`ProjectRepository.getProjects` (sort: best match) plus the matches cache	
3	Show the reason with every score	`matchedSkills`, `missingSkills`, `explanation`	A score is never shown alone
4	Explain AI	`llmExplain`	Natural language
5	Cross-skill match (skills close to the required ones)	`llmExplain.complementarySkills`	The matching logic for "close" skills is not yet defined (Section 13)
6	Skill gap	`missingSkills[]`	Compare user skills with project skills
7	Short learning tip per missing skill	`llmExplain.learningTips[]`	Fallback text if the LLM fails
8	Rate the recommendation: helpful or not (and a reason when rejected)	`feedback/{id}` (`helpful`, `reason`)	
9	Rate explanation clarity, 1 to 5	`feedback/{id}.clarity`	
10	Save the recommendation in Match History	`matches/{uid}/items/{projectId}`	The cache doubles as history
11	Show Match History	Query `matches/{uid}/items`	Needs a date field to order by (Section 13)
12	Fallback when the LLM is slow or down	`llmExplain` returns `source: "fallback"` or `cached`	A short notice, never an empty screen (NFR-11)

Targets: response time of 5 seconds or less; 60% or more of recommendations rated helpful by testers; average explanation clarity of 4 out of 5 or more.

Feature 6 — Project Discovery

Goal: the user finds the right project (or developer) with the least effort.

Screens (7): Discover Home (Recommended, Trending / Newest, search), Discover toggle (Projects / Developers), Project Filters, Search Results, Discover Developers, Developer Public Profile, Saved Projects.

#	Operation	API / implementation	Notes
1	Browse recommended projects	`ProjectRepository.getProjects` plus `matches` cache	Recommended, Trending, Newest
2	Switch between Projects and Developers	UI state	Developers is Should
3	Keyword search	Client-side filter over title, description and tech stack	Firestore has no full-text search
4	Natural-language search ("I want a Flutter project for beginners")	None defined	Could
5	Apply or reset filters	`getProjects(filters)`	See the filter table
6	Sort results (Best match, Newest)	`getProjects(sort)`	`createdAt` descending for Newest
7	Open project details	`projects/{id}`	Feature 8
8	Save or unsave a project	`users/{uid}/saved/{projectId}` set or delete	Requires login
9	Express Interest	Storage not defined in the schema	Separate action from Save and Request to Join (Section 13)
10	Show the Saved list	Stream `users/{uid}/saved`, then load the projects	
11	Browse developers	Query `users`	Should
12	Open a developer's Public Profile	`ProfileRepository.get`	
13	From a profile: Invite to project or Start chat	Invitation / chat (Should)	Requires login

Filters

Filter	Field	Where it runs
Status (Open / Full)	`status`	Firestore
Project purpose (Learning / Real)	`purpose`	Firestore
Difficulty	`difficulty`	Firestore
Tech stack	`techStack`	Firestore `array-contains-any` (only one array filter is allowed per query)
Roles needed	`roles[].name`	Client-side (the data set is small)
Team size	`teamSize`	Client-side
Project type, Track	No matching field in the schema	Open decision (Section 13)

Pagination: `limit(10)` with `startAfter`. The demo data set is 15 to 20 projects.

Feature 7 — Community — [Unapproved] (Should / Could)

Goal: a place for developers to ask questions, share ideas and find teammates.

Screens (3): Community Feed, Create Post, Post Details (with comments).

The plan only requires a placeholder screen for the Community tab. If time allows, the minimum is **Feed (read) plus Create Post**. Guests can read the feed. Any interaction requires login. No schema is frozen for this feature, so the collections below are **[Proposed]**:

- `posts/{postId}`: `authorId`, `text`, `tags[]`, `likeCount`, `commentCount`, `createdAt`
- `posts/{postId}/comments/{id}`: `authorId`, `text`, `createdAt`
- `posts/{postId}/likes/{uid}`
- `reports/{id}`: `postId`, `reporterId`, `reason`

#	Operation	API / implementation	Notes
1	View the Feed	Query `posts` (newest first)	Guest allowed
2	Create a post (text and tags)	Create `posts/{postId}`	Requires login
3	Like	Set or delete `posts/{postId}/likes/{uid}`	
4	Comment	Create `posts/{postId}/comments/{id}`	
5	Open a post	Read `posts/{postId}` and its comments	
6	Edit or delete your own post	Update or delete `posts/{postId}`	Author only
7	Report content	Create `reports/{id}`	
8	Open the author's profile	`ProfileRepository.get`	
9	Start an individual chat with the author	Chat (Should)	
10	Filter the Feed by tags	Query `posts` with `tags`	

Feature 8 — Project Details

Goal: the user understands the project, sees whether it fits, and applies.

Project purpose. Every project has a purpose chosen at creation. It is shown as a clear badge on the details page and on Create Project, but never as a hidden field.

Type	Meaning	Related fields
Learning	Training project, suitable for beginners	Beginner-friendly, optional mentoring, no strict deadline
Real-world	A real product or active open-source project	Expected commitment (hours per week), required experience level, deadlines

Related fields: expected commitment, minimum level, mentoring available (yes or no), expected end date. AI Match takes the purpose into account (a beginner is offered Learning projects first).

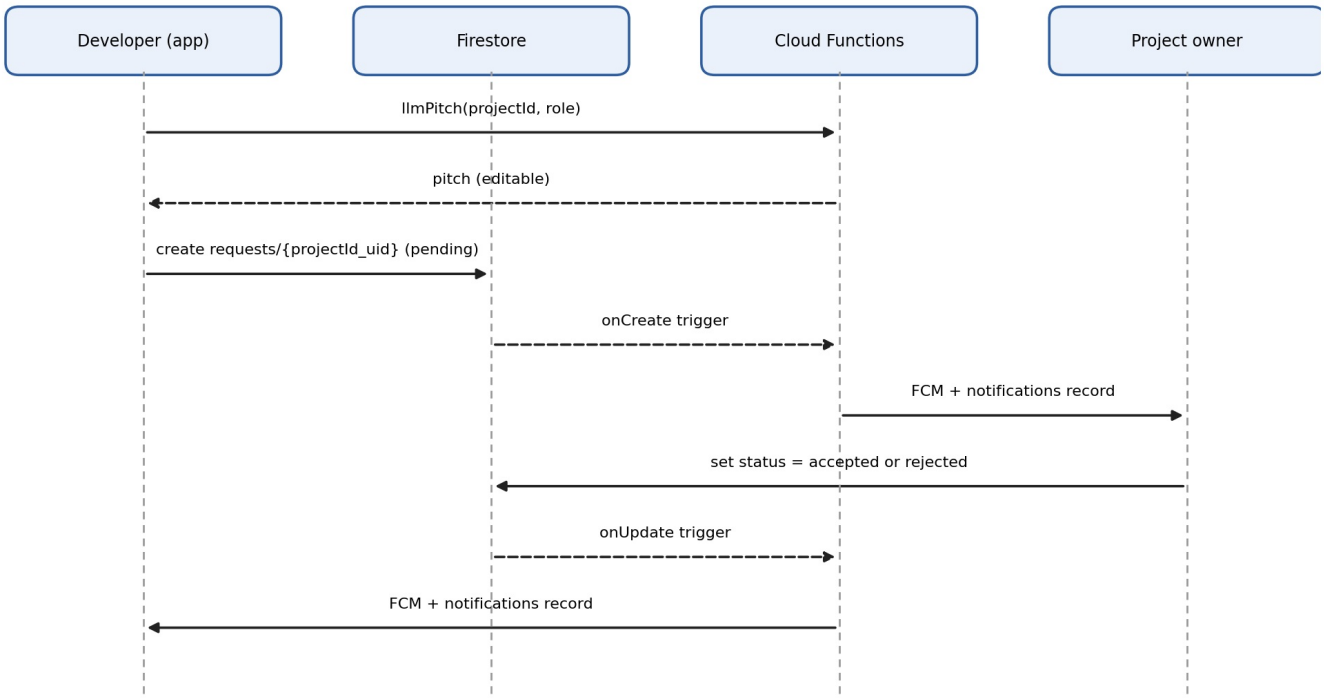
Screens (3): Project Details (title, description, purpose badge, tech stack, difficulty, roles, current team, "Why this fits you", Skill Gap, owner, expected commitment, Save, Express Interest, Request to Join), Request to Join (pick a role, AI-generated message, editable), Request Sent (confirmation and status).

Save Project, Express Interest and Request to Join are **three separate actions**. All three require login.

#	Operation	API / implementation	Notes
1	Show project information	`projects/{projectId}`	Guest allowed
2	Show the purpose badge and its details	`projects/{projectId}` (`purpose`, commitment, mentoring)	
3	Show required roles and the current team	`projects.roles[]`, `teams/{projectId}/members`	
4	Show the match and explanation	`matches/{uid}/items/{projectId}`	Links to the AI Match screen
5	Save	`users/{uid}/saved/{projectId}`	

6	Express Interest	Not defined in the schema	Section 13
7	Request to Join	Opens the Request to Join flow	`requireAuth` first
8	Generate a Collaboration Pitch with AI	`llmPitch`	
9	Edit the message	Local	The user always reviews it
10	Send the request (once only)	Create `requests/{projectId}_{uid}` with `status: pending`	The composite ID and the rules block duplicates
11	Show request status (Pending, Accepted, Rejected)	Stream the request document	Updates live
12	Share the project link	Shared link / deep link (Should)	Link format is open (Section 13)

Sequence: Request to Join



Feature 9 — Create Project

Goal: the owner creates a project quickly (manually or with AI), publishes it, and is then alerted to missing roles.

Screens (8): Create Entry (Manual or AI-assisted), Describe Your Idea (two sentences), AI Project Setup (description, tech stack, difficulty, suggested roles and tasks, all editable), Project Details Form (title, description, Project Purpose, expected commitment), Tech Stack and Required Skills, Role Generator (Frontend, Backend, QA, ...), Review and Publish (with Team size and Difficulty), Publish Success (with a Find Missing Roles button).

State: `CreateProjectCubit` keeps the whole form in one state. Steps: details → tech stack → roles → review and publish. A project is saved as `draft` and becomes `open` only when the owner publishes. **There is no automatic publishing.** Review and Publish shows the project purpose (Learning or Real) clearly before publishing.

#	Operation	API / implementation	Notes
1	Choose manual or AI creation	Local	
2	Write the idea description	Local	Two sentences
3	Generate the setup with AI	`llmProjectSetup`	Fills the form; every field stays editable
4	Edit title and description	Local form state	
5	Choose the project purpose (Learning or Real)	Local; saved as `purpose`	
6	Choose or edit the tech stack	Local; picked from `skills_catalog.dart`	
7	Confirm the required skills	Local	
8	Generate roles with AI	`llmRoles`	
9	Edit, add or delete roles	Local; `roles[{name, requiredSkills[], count, filled}]`	
10	Set team size, difficulty and project type	Local	
11	Save a draft	Create or update `projects/{id}` with `status: draft`	

12	Publish (owner approval only)	Update `projects/{id}.status` to `open`	
13	After publishing: Missing Role Alert and Find Suitable Developers	Alert when `filled < count`. Developer search queries profiles and ranks them with `MatchingService` (top 20)	Find Suitable Developers is Should
14	If the AI fails: manual input with a clear message	Catch `deadline-exceeded / unavailable`	UC-04 alternate path

---

Feature 10 — My Teams

Goal: the team's day-to-day workspace: members, chat and tasks.

Screens (7): My Teams List, Team Workspace (Project Info, Members, Team Chat, Tasks), Team Members (members, roles and missing roles), Team Chat, Task Board (To Do, In Progress, In Review, Done), Task Details, Create / Edit Task.

Notes

- **Tasks are Must. Team Chat is Should.**
- Task status changes through a picker, not drag and drop (simpler and faster to build).
- Permissions: only team members read and write (`isMember` from `memberIds`).
- Offline: Firestore saves the operation locally and sends it later. The app shows a simple banner (UC-05).
- The workspace uses the seed teams until request acceptance works end to end.

#	Operation	API / implementation	Notes
1	Show the teams list	Query `teams` where `memberIds` contains the uid	
2	Open the Workspace	`teams/{projectId}` plus `projects/{projectId}`	
3	Show members and their roles	`teams/{projectId}/members`	
4	Send and receive team messages in real time	Proposed: `teams/{projectId}/messages/{id}` (`senderId`, `text`, `createdAt`)	Should; [Unapproved]
5	Create a task (description, assignee, deadline, priority)	Create `teams/{projectId}/tasks/{taskId}` with `status: todo`	Date picker for the deadline
6	Assign a task to a member	Set `assigneeId` (chosen from the members)	
7	Update task status	Set `status`	`todo`, `inProgress`, `inReview`, `done`
8	Edit or delete a task	Update or delete the task document	
9	Filter tasks by assignee or status	Stream `tasks`, filter by assignee and status	
10	Missing Role Alert inside the team	Roles where `filled < count`	
11	Leave the team	Not in the plan. Proposed: a transaction that removes the user from `memberIds` and `members`, and lowers `filled`	[Proposed]
12	Notify the assignee on assignment or update	`onTaskWritten` trigger (FCM)	Opens the task from the notification

Example: task document

```
{
  "title": "Build the profile screen",
  "description": "Implement the Profile form UI with validation.",
  "assigneeId": "u_abc",
  "status": "inProgress",
  "priority": "high",
  "deadline": "2026-11-12",
  "createdBy": "u_owner"
}
```

Feature 11 — My Projects

Goal: manage projects from the point of view of the role (Owner or Member), and handle join requests and invitations.

My Projects versus My Teams: *My Projects* is the project itself (status, requests, editing). *My Teams* is the daily work (Workspace, Chat, Tasks). The team may decide to merge them (Section 13).

Screens (8): Ongoing, Invited, Previous, Project Overview (Owner), Incoming Requests, Applicant Details (profile, matched skills, pitch message, CV), Invite Developer, Invitation Details.

Tab queries

- **Ongoing:** projects I own or belong to, with a status other than `completed` (two queries merged: `ownerId` equals my uid, and projects of the teams where I am a member).
- **Invited:** invitations sent to me (Should).
- **Previous:** projects with `status: completed`.

Each project shows an **Owner** or **Member** badge and its status.

#	Operation	API / implementation	Notes

1	Switch between Ongoing, Invited and Previous	Queries above	
2	Open the Project Overview	`projects/{id}`	Owner view
3	Edit the project	Update `projects/{id}`	Owner only
4	Close or archive the project	Set `status` to `completed`	Moves it to Previous *(proposed mapping)*
5	View incoming requests	Query `requests` where `ownerId` equals my uid and `status` is `pending`	
6	Accept a request (applicant joins the team and is notified)	One transaction or batch (see below)	Notification is sent by the Function, not the app
7	Reject a request (notify and close it)	Set `requests/{id}.status` to `rejected`	`onRequestStatusChanged` notifies the applicant
8	View applicant details	`users/{applicantId}`, `MatchingService` for matched skills, `requests.pitch`, CV link	CV access mechanism is open (Section 13)
9	Invite a developer	Proposed: create `invitations/{id}`	Should; [Unapproved]
10	Accept or decline an invitation	Update the invitation	Should
11	Cancel an invitation	Delete or update the invitation	Should
12	Change project status (Open, Full, Completed)	Update `projects/{id}.status`	
13	FCM for these events (delivered in 10 seconds or less)	`onRequestCreated`, `onRequestStatusChanged`	**NFR-02**

Accepting a request must be atomic, so the data is never left half-updated:

1. Set `requests/{id}.status` to `accepted`.
2. Create `teams/{projectId}` if it does not exist, and add the applicant to `memberIds` and to `members/{uid}`.
3. Increase `filled` for the requested role in `projects/{projectId}.roles`.
4. If every role is filled, set the project `status` to `full`.

The owner should also be a member of the team, so the `isMember` rule covers the owner's access to tasks *(proposed)*.

Feature 12 — Prompt Evaluation

Goal: measure the quality of AI answers and improve the prompts based on real data. This is required by the course brief.

Screens (2):

1. **Evaluate AI Suggestion** appears after any AI output (recommendation, project setup, pitch): helpful or not helpful, a clarity rating from 1 to 5, and an optional reason.
2. **Prompt Evaluation Report** is an internal screen for developers and the demo. It shows a table comparing prompt versions and their results.

#	Operation	API / implementation	Notes
1	Collect the user's rating after each AI output	Feedback bottom sheet	
2	Store the rating with the prompt version	Create `feedback/{id}` (`uid`, `projectId`, `helpful`, `clarity`, `reason`, `promptVersion`)	`promptVersion` comes from the `matches` document
3	Compare prompt versions	Aggregate `feedback` by `promptVersion`	
4	Compute the metrics: relevance and explanation clarity	Relevance is the share of "helpful" ratings. Clarity is the average of 1–5 ratings	
5	Run each version on a fixed test set of sample profiles and projects	Offline test script or manual run	
6	Record the results in a table in the documentation	Documentation	The deliverable required for submission
7	Activate the best version	Set `config/prompts.activeVersion` (or Remote Config)	Functions read it on every call

Report columns: prompt version, number of ratings, helpful rate, average clarity.

Targets: 60% or more of recommendations rated helpful; average clarity of 4 out of 5 or more.

Adaptive Matching (the app learning from rejections) is outside the plan. The feedback data collected here allows it as future work.

Feature 13 — Chatbot — [Unapproved] (Could)

Goal: an in-app assistant that answers questions and guides the user. It is in the idea list (feature 57) but **not in the approved plan**. It is built only after the Must features are complete, and it is locked in the Pro flavor.

Screen (1): Chatbot, a conversation with suggested prompts.

#	Operation	API / implementation	Notes
1	Open the chat (floating icon, or from Discover or Profile)	Local navigation	

1	Open the chat (floating icon, or from Discover or Profile)	Local navigation	
2	Send a question and receive a reply	`chatbotAsk`	
3	Suggested prompts ("What skills am I missing?", "Suggest a project for me to learn")	Local list	
4	Replies based on the user's context (profile and skills)	`chatbotAsk` loads the context server-side	
5	Deep link to a project or screen from inside a reply	`deepLink` in the response	
6	Clear the chat history	Local	History stays in the app by default
7	Per-user request cap (cost control)	Counter enforced in the function	Fails with `resource-exhausted`
8	Fallback if the LLM is down	`source: "fallback"`	A clear message, never an empty screen
9	Rate the reply (helpful or not)	`feedback` record	

---

9. Cross-cutting services

9.1 Notifications (FCM client and Notifications screen)

- **Client:** `firebase_messaging` with `flutter_local_notifications`. The app saves the device token to `users/{uid}.fcmTokens[]` and refreshes it on change. It handles foreground, background and terminated states and opens the right screen from the `type` and `refId` payload.
- **Notifications screen (2 screens):** a list grouped as Today and Earlier, with an empty state. It streams `notifications/{uid}/items`, supports mark as read, and shows an unread badge on the bell icon.
- **Navigation:** Notifications are a **bell icon in the top app bar**, never a bottom-navigation tab. The five bottom tabs are Discover, Community, My Projects, Messages and Profile.
- **Fallback:** if push delivery is unreliable, the in-app notification list is the backup (risk R8).

9.2 Settings and Account

- Logout and Delete account (see Feature 2).
- Connected Accounts: connect or disconnect GitHub (Feature 4), Behance link.
- Delete account must also delete the user's data.

9.3 Free and Pro flavors

- Two flavors (`free`, `pro`) are built from the same code (`main_free.dart`, `main_pro.dart`).
- Proposed split: **Free** has basic profile, project discovery, basic matching and Community. **Pro** has advanced AI matching, skill analysis, AI Team Builder, growth analytics and advanced insights.
- The team must pick at least one real feature that is locked in Free and shown in the demo (for example the detailed Skill Gap). A locked feature opens an **upgrade information screen** only.
- **There are no real payments, no invented prices and no usage limits** in the plan. Use the text "Plan benefits to be confirmed".
- A flavor is a build-time setting. It is not a security boundary, so it must not be the only protection for anything sensitive.

9.4 Offline behavior

Firestore offline persistence is on. Task changes made offline are queued and sent later. The app shows a simple offline banner. The alternative is to ask the user to reconnect (UC-05).

---

10. Security rules and privacy

Principle: least privilege. A user writes only to their own data.

Resource	Read	Write
`users/{uid}`	See open decision 4	Owner
`users/{uid}/githubData`, `detectedSkills`	Owner	`githubProxy` (githubData); owner updates `status` (detectedSkills)
`users/{uid}/saved`	Owner	Owner
`private/{uid}`	**No client access**	`githubProxy` only
`projects`	Everyone, including Guests	Owner
`requests`	Applicant and project owner	Applicant creates once (composite ID blocks duplicates); owner changes `status`
`teams`, `tasks`, `members`	Team members	Team members
`matches/{uid}`	Owner	Owner and `llmExplain`
`feedback`	Author (aggregation for the report: to be decided)	Author

`prompts`, `config`	Authenticated users or Functions	Admin only
`notifications/{uid}/items`	Owner	Functions create; the owner updates `read`
Storage `cvs/{uid}`	Owner; project owner for the project the user applied to	Owner

Testing: use the Firebase Emulator or a manual checklist (scenario, expected result, actual result). Examples: a user tries to edit another user's data; a Guest tries to write. The target is **100% of the checklist passing** (NFR-03).

Other protections

- No keys or secrets in the app code or the repository (NFR-04). Secrets live in Cloud Functions secrets.
- Runtime permissions are requested only when needed, following the Android security model (NFR-05).
- Code obfuscation in the release build (NFR-06): `flutter build appbundle --release --obfuscate --split-debug-info`, and the symbol files are uploaded to Crashlytics.
- GitHub is connected with least privilege and the user can disconnect at any time. Only the data needed for matching is used (NFR-07).
- HTTPS for all connections. Tokens are never stored in the app.

11. Error handling matrix

Scenario	Source	App behavior
Guest taps a protected action	`requireAuth`	Login Required sheet; return to the same screen after login
Wrong password, unverified email, no network	Firebase Auth	Clear message; Resend verification button
Duplicate join request	Firestore rules reject the second create (`permission-denied`)	"You already sent a request"
GitHub rate limit or API failure	`githubProxy`: `resource-exhausted` or `unavailable`	Clear message; continue with manual data
GitHub not connected	`githubProxy`: `failed-precondition`	Offer to connect
LLM slow or down (explanation)	`llmExplain`: `source: "fallback"`	Rule-based or cached text with a short notice
LLM slow or down (pitch, project setup, roles)	`deadline-exceeded` / `unavailable`	Manual input with a clear message
Daily AI limit reached	`resource-exhausted` or `fallbackReason: "daily_limit"`	Fallback content and a short notice
Profile too incomplete to match	`MatchingService`	"Complete your profile first" with a button to Profile Completion
Task change while offline	Firestore offline queue	Offline banner; sent later
Missing Firestore composite index	Firestore error	Fix by adding the index to `firestore.indexes.json`
Deep link to a removed or cancelled item	Router validation	A message about the status (for example, "This invitation was cancelled")
Unexpected failure	`internal`	Generic message; logged in Crashlytics

12. Performance targets

Metric	Target	Source
Recommendations load time	5 seconds or less	**NFR-01**
FCM delivery time	10 seconds or less on real devices	**NFR-02**
Crash-free sessions	95% or more	NFR-10 (Crashlytics)
Security rules checklist	100%	**NFR-03**
Recommendations rated helpful	60% or more	Product KPI
Explanation clarity	4 out of 5 average or more	Product KPI
Collaboration requests accepted	40% or more	Product KPI
Profiles completed by testers	80% or more	Product KPI

13. Open decisions

These are the API-related gaps found while writing this document.

1. **Function contracts.** All input and output shapes in Section 4, the function region and the trigger names are **[Proposed]**. Confirm them with Mohraeel.
2. **AI daily limit.** The value and the Firestore path of the counter are not set.

3. **Schema additions needed by features:** `photoUrl`; `minLevel` and `expectedEndDate` on projects; `behanceUrl` (in Core Features but not in the plan's schema); a date field on `matches` items so Match History can be ordered; a marker for skills that came from GitHub; a `kind` field on `feedback` so pitch and project-setup outputs can be rated (today `feedback` only has `projectId`); storage for Express Interest; a field for the optional free-text goal on the profile.
4. **Public and private user data.** Guests and other users read public profiles, but the plan stores `fcmTokens[]` inside `users/{uid}`. Split public fields from private ones, or move tokens to a private document, so tokens are not readable by others.
5. **Account deletion.** Decide how the user's data is deleted (client batch, or a Cloud Function triggered by account deletion).
6. **Skill inference location.** Decide whether the rule-based step runs in the app or in the function.
7. **CV access for project owners.** Decide how the owner of a project reads an applicant's CV.
8. **Project type and Track filters.** The schema has only `purpose` (Learning or Real). The design also lists Portfolio and Open Source, and Core Features lists a Track filter.
9. **Owner as a team member**, so the `isMember` rule covers the owner.
10. **Cross-skill match.** The logic for "close" skills is not defined.
11. **Schemas for Community, team chat, individual chat and invitations** (Should / Could).
12. **Chatbot scope.** Decide whether it ships at all (Core Features: out of the MVP).
13. **Shared link and deep link format.**
14. **Leaving a team.** The behavior (and whether a full project reopens) is not defined.

Also open in Core Features: Behance as link-only or data import; whether My Teams and My Projects merge into one tab; whether the CV is analyzed by AI (future work); the Guest mode scope; and confirming the Firebase Blaze plan early, because Cloud Functions need it.

---

14. Appendices

A. Enumerations

Name	Values
Track	Mobile, Web, Backend, Data, AI, UI/UX, QA, DevOps (the list may grow)
Level	Beginner, Junior, Intermediate, Advanced
Project purpose	`learning`, `real`
Project status	`draft`, `open`, `full`, `completed`
Request status	`pending`, `accepted`, `rejected`
Task status	`todo`, `inProgress`, `inReview`, `done`
Detected skill status	`pending`, `accepted`, `rejected`
Notification type **[Proposed]**	`request_received`, `request_accepted`, `request_rejected`, `task_assigned`, `task_updated`
`llmExplain.source` **[Proposed]**	`llm`, `cached`, `fallback`
Difficulty and task priority	Values not yet defined. Proposed: same four-level scale as Level for difficulty, and low / medium / high for priority

Stored casing (lowercase or capitalized) must be frozen in `docs/schema.md`.

B. Traceability

Module	User stories	Use cases	Functional requirements	Priority
Authentication	US-01	UC-01	FR-01, FR-02	Must
Profile and GitHub	US-02, US-03, US-04	UC-01	FR-03, FR-04, FR-05	Must (Behance: Should)
Discovery	US-05	UC-02	FR-06	Must
AI Matching	US-06 to US-09, US-25	UC-02	FR-07 to FR-11	Must
Projects	US-10 to US-13	UC-04	FR-12 to FR-14	Must (AI assistance: Should)
Collaboration	US-14, US-15, US-17	UC-03	FR-15, FR-16	Must (invitations and links: Should)
Notifications	US-16	UC-03	FR-17	Must
Tasks	US-18	UC-05	FR-18	Must
Communication	US-19, US-20, US-24	none	FR-19, FR-20, FR-25	Should / Could
Utilities	US-21, US-22	none	FR-21, FR-22	Could
Release and UX	US-23	none	FR-23, FR-24	Must (animations: Should)

C. Out of scope (future work)

Not documented as APIs because the approved plan excludes them: iOS and web versions; real payment handling for Pro; Developer Digital Twin; GitHub skill verification, skill recency and evolution, hidden skill detection; GitHub activity score; AI Team Builder and find-missing-person search; AI learning path and growth dashboard; Adaptive Matching and collaborative filtering; project health, risk and bottleneck detection; chat summary and action items; CI and issue tracker integration; AI analysis of the CV; a richer community area.

D. Changelog

Version	Date	Change
0.1	27 Oct 2026	Initial draft covering all 13 core features